PATENT REVIEW

Hierarchical Bandit-Guided Scientific Discovery

Language models generate programs that construct scientific solutions. Evaluated outcomes guide search and improve the generating model.

Technical architecture, advantageous effects, and patent value

Contents

- Idea Compliance Check
- Background
  - LLM-Based Scientific Discovery
  - Prior Art: Published Discovery Systems
  - Technical Problems in Existing Approaches
- Technical Solution
  - Hierarchical Discovery Pipeline
  - Allocation and Policy Training
  - Dependency-Aware Execution
- Distinguishing Features and Design Rationale
- Advantageous Effects of the Idea
- Patent Value
  - Detectability and Avoidability
  - Commercial Applications and Value

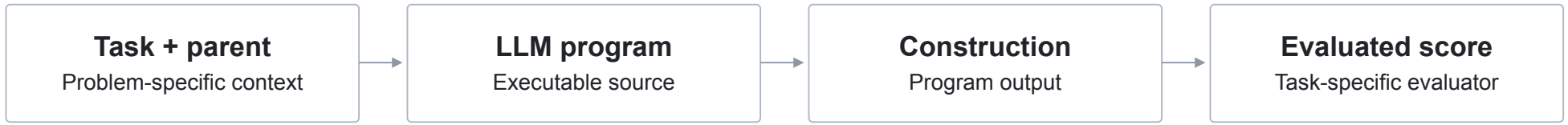
Idea Compliance Self-Check

No.	Self-check item	Declaration
1	Does the idea involve another party's non-public information from technical collaborations, procurement, supplier or customer communications, or another organization?	No
2	No non-public software algorithms, designs, layouts, or confidential technical solutions of a third party are used without authorization.	Confirmed
3	No third-party non-public information forms the basis of an improvement to the technology described in that information.	Confirmed
4	No non-public third-party information obtained by disassembly or reverse engineering is used in the patent application.	Confirmed
5	The idea does not involve non-public information of a former employer or work products belonging to that employer.	Confirmed

Inventor declarations. Use the applicable official submission checklist.

Background: LLM-Based Scientific Discovery

A large language model (LLM) writes a program that constructs a candidate solution. A task-specific evaluator checks the construction and assigns its score.



Search with a fixed model

The search reuses promising solutions as context for new programs. The model's weights stay unchanged.

Search with reinforcement learning

Evaluation also supplies a learning signal. The LLM updates its weights to favor programs with stronger outcomes.

Parent: a scored starting solution. **Rollout:** one generated program. **Archive:** retained solutions and their ancestry.

Success means finding a better valid construction, even if many attempts fail.

Prior Art: Published Discovery Systems

Published work	Relevant technical approach
FunSearch Romera-Paredes et al., Nature, 2024 Published online December 2023 Mathematical discoveries from program search with large language models doi:10.1038/s41586-023-06924-6	A pretrained LLM proposes programs. An evaluator scores them, and an island-based evolutionary population supplies examples for subsequent generation.
AlphaEvolve Novikov et al., 2025 AlphaEvolve: A coding agent for scientific and algorithmic discovery arXiv:2506.13131	LLMs modify code using an evolutionary search loop and evaluator feedback. The system retains useful programs and improves them over repeated iterations.
TTT-Discover Yuksekgonul et al., 2026 Learning to Discover at Test Time arXiv:2601.16175	Reinforcement learning adapts the LLM to one discovery problem. An entropic objective favors high rewards, and a PUCT-based archive selector reuses promising solutions.

Technical Problems in Existing Approaches

01 **One generation combines algorithm selection and implementation**

The LLM chooses an approach while writing the program. Additional samples can repeat the same approach without explicitly testing alternatives.

02 **Reinforcement learning can favor copies of an existing solution**

Reproducing an existing construction can still earn a high reward. Reinforcing these outputs can make the LLM generate more copies and fewer distinct constructions.

03 **A fixed sampling split cannot adapt to early evaluation results**

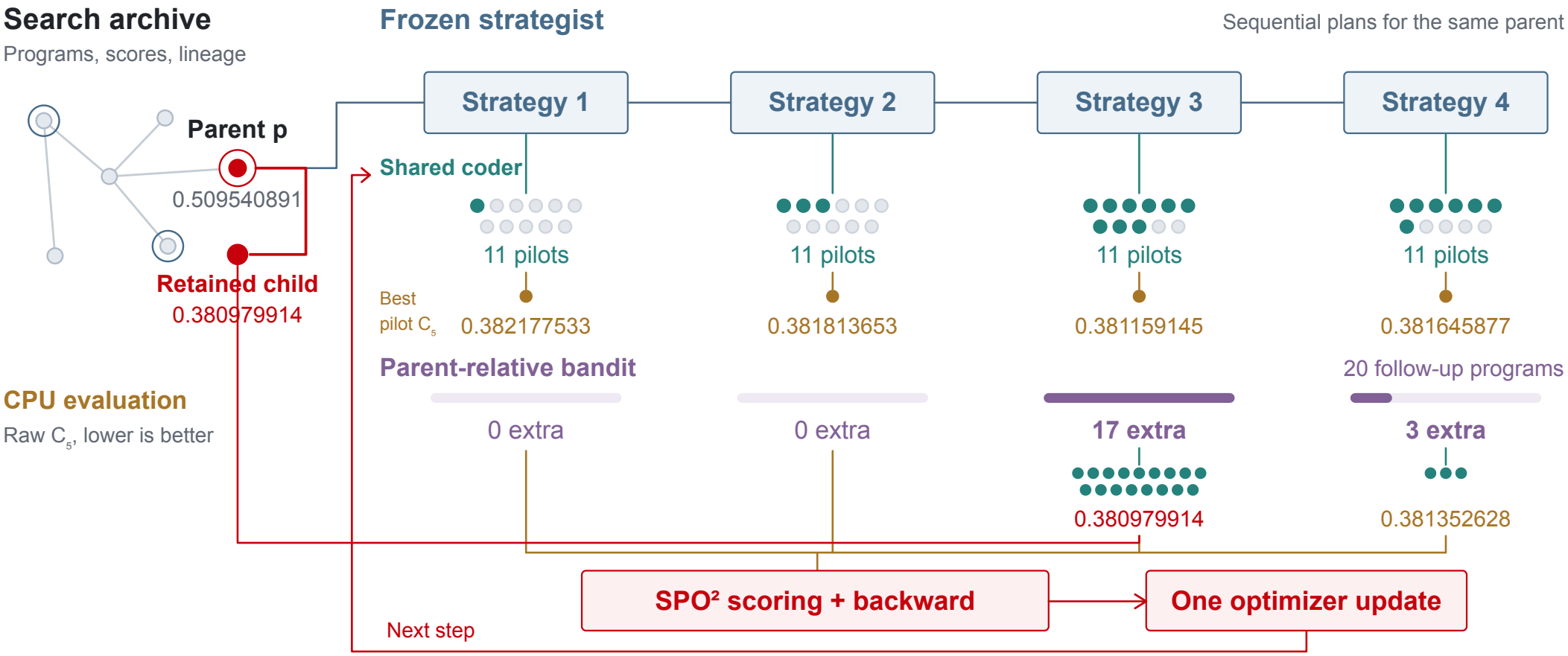
Equal sampling continues after early results indicate that some directions are more promising. The remaining budget cannot shift toward those directions.

04 **Group-relative training can discard useful reward information**

When all programs in a group receive the same reward, their group-relative advantages are zero. This removes the reward-based update even if that reward improves on earlier results.

Technical Solution: Hierarchical Discovery Pipeline

Retain children and update the shared coder



Selected children enter the archive. One optimizer update prepares the coder for the next step.

Technical Solution: Allocation and Policy Training

Parent-relative bandit allocation

- Generate H pilot programs for each of S strategies.
- Estimate each plan's chance of improving on its parent and the size of successful gains.
- Allocate B follow-ups by their expected effect on the best sampled reward, accounting for uncertainty.

$$Z_{k,j} = \max(R_{k,j} - R_p, 0)$$

$R_{k,j}$: reward of pilot j for strategy k. R_p: parent reward.
$Z_{k,j}$: improvement over the parent, set to zero for invalid outputs.

$$a_k \in \mathbb{Z}_{\geq 0}, \quad \sum_k a_k = B \quad N_{\text{parent}} = SH + B$$

a_k: follow-ups for strategy k.
N_{parent}: total planned programs for that parent.

Historical-baseline training (SPO²)

- Higher rewards receive exponentially larger weights.
- Compute every advantage against the same moving average of earlier transformed rewards.

$$G(r) = e^{\beta r}, \quad \beta > 0$$

$$A_{t,i} = \frac{1}{\beta}\left(\frac{e^{\beta r_{t,i}}}{v_{t-1}} - 1\right)$$

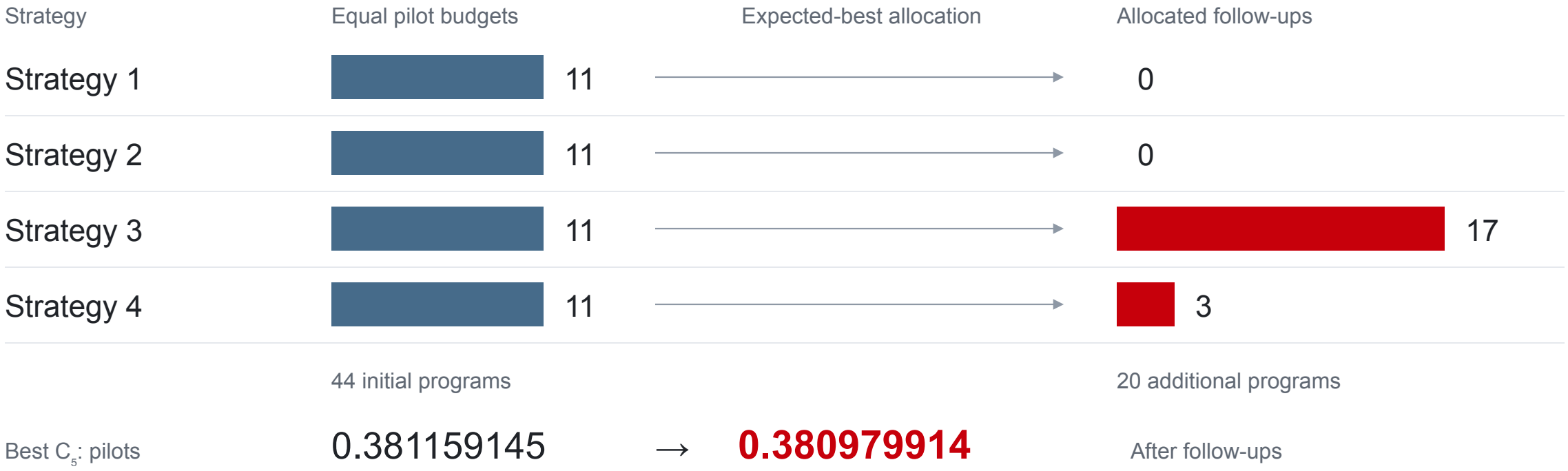
v_{t-1}: moving average of earlier G(r). β: strength of reward shaping.
$r_{t,i}$: reward. $A_{t,i}$: training weight for rollout i at step t.

Positive A reinforces a program. Negative A discourages it. A tied group can still receive a nonzero weight relative to earlier rewards. Hold β and the baseline fixed within the step, then update the baseline.

R and r denote larger-is-better training rewards. Scientific plots show the raw task score, minimized for the Erdős problem.

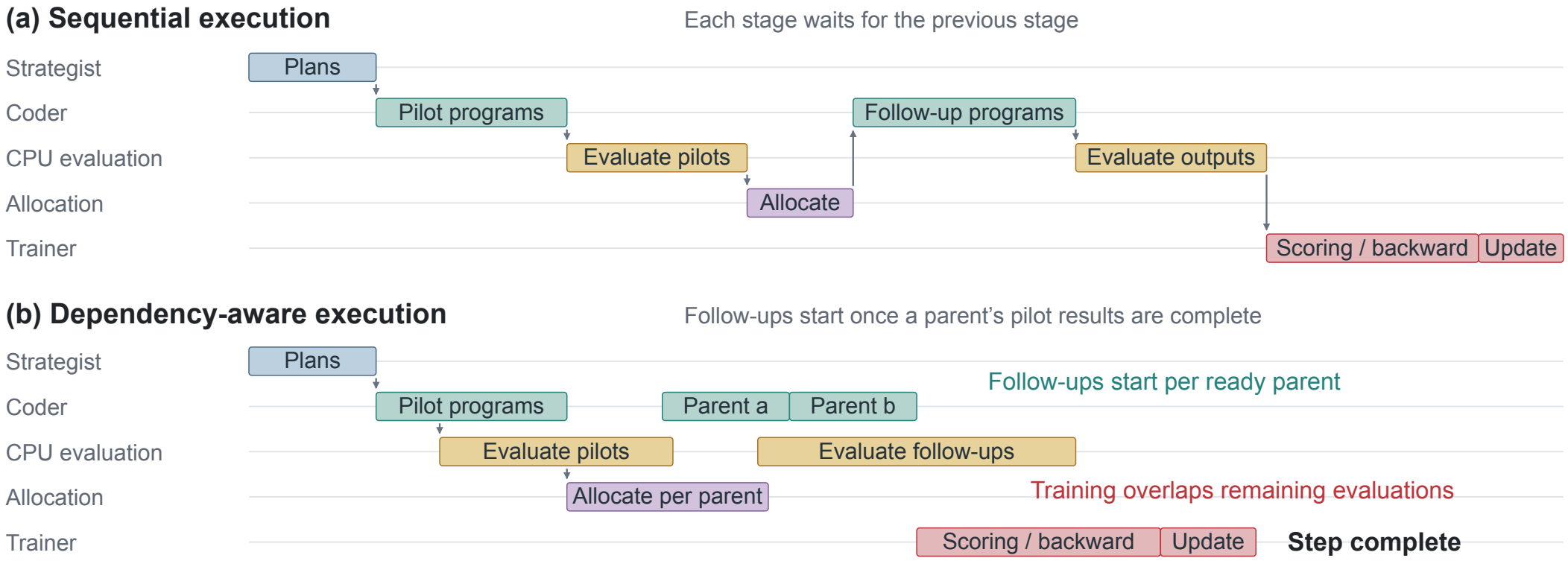
Observed Allocation: Equal Pilots, Targeted Follow-ups

One parent has a budget of 64 programs. Generate 11 pilots for each of four strategies, then use their results to allocate the remaining 20 programs.



Erdős minimum-overlap task. C_5 is the evaluated bound, lower is better.

Technical Solution: Dependency-Aware Execution



Evaluate programs as they arrive. Once generation ends, train on evaluated programs while CPU evaluation continues. **Keep parameters fixed until the step's final optimizer update.**

Distinguishing Features and Design Rationale

Proposed mechanism	Operation and technical purpose
Frozen strategist and shared learning coder	The strategist proposes alternative plans. One coder learns from programs across those plans, separating algorithm selection from implementation learning.
Parent-relative allocation objective	Estimate both the probability of beating the parent and the gain when successful. This lets rare, large improvements influence the next sampling decision.
Pilots followed by adaptive budgets	Evaluate equal pilot batches before assigning follow-ups. Their outcomes determine where the remaining fixed program budget goes.
Score-based learning with SPO²	Retain differences between valid scores and compare transformed rewards against one historical baseline. Even tied groups can receive a nonzero learning signal.
Execution with one final policy update	Accumulate gradients on ready outputs while other evaluations finish. Keep parameters fixed until the step's required results and gradients are complete.

The proposed contribution is the coordinated control of planning, sampling, and learning. Archive reuse and lineage-aware PUCT are supporting mechanisms already described in TTT-Discover.

Advantageous Effects: Search and Learning

Effects supported by the mechanism

- **Explore explicit alternatives:** condition program generation on different plans while the shared coder improves its implementations.
- **Respond to early evidence:** move follow-up samples between plans after pilot evaluation, without increasing the total program budget.
- **Preserve useful reward information:** distinguish stronger valid solutions and compare their rewards with a shared history.

One observed exploration comparison

Erdős task, Qwen3-8B coder, step 19
192 attempts per step in each run

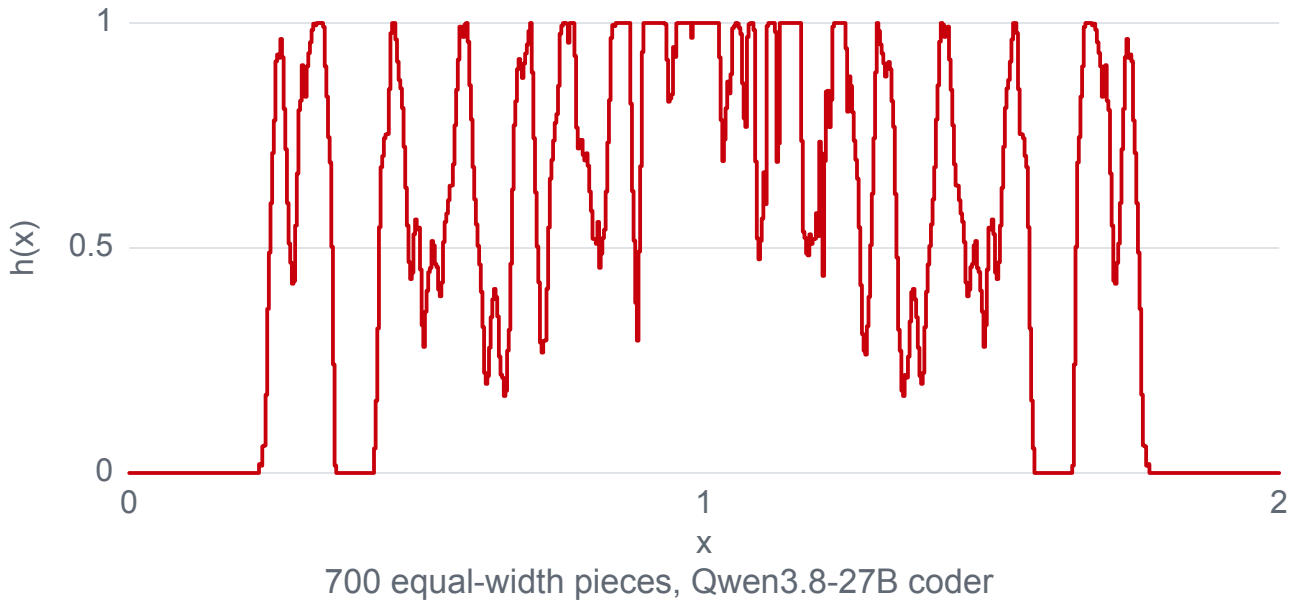
Outputs (% of attempts)	With plan	Without
Distinct valid non-parent constructions	**14.1%**	**1.0%**
Copies of selected parents	**2.1%**	**92.2%**

The plan-conditioned run retained more distinct constructions at this step.

One run per condition. Equal attempt counts do not establish equal total compute or isolate causality.

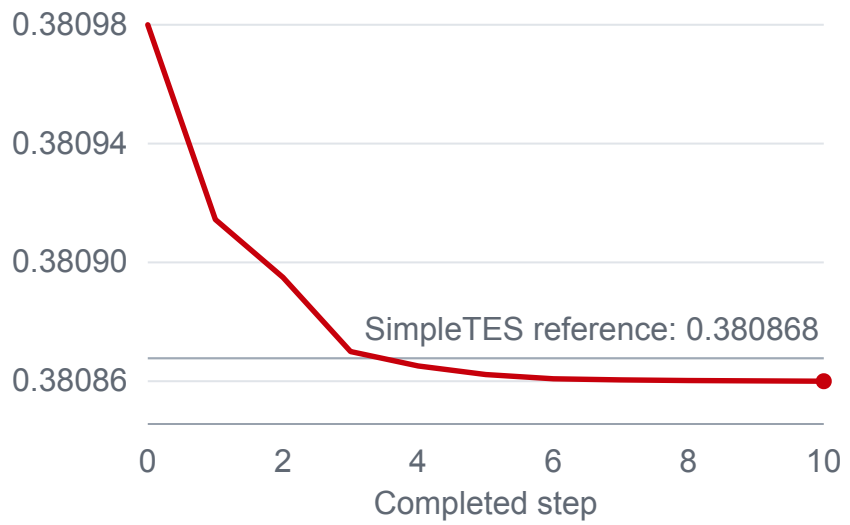
Advantageous Effects: Evaluated Construction

Best saved construction



700 equal-width pieces, Qwen3.8-27B coder

Best-so-far C_5 bound



$C_5 \leq 0.380860634340$

The full pipeline found a construction below the historical reference of 0.380868. Re-evaluating its saved array reproduces the bound shown here.

Patent Value: Detectability

Trace each parent through its plans, scored programs, budget decisions, and policy update.

Mechanism	Evidence to inspect	What the evidence can establish
Separate model roles	Plan prompts, coder prompts, model versions, trainable-parameter lists	Plans condition generation, the strategist stays fixed, and the shared coder receives updates.
Pilot-dependent allocation	Parent and plan IDs, pilot scores, allocation records, follow-up counts	Pilot outcomes feed a parent-relative allocation rule and determine the fixed follow-up budget split.
Historical-baseline learning	Reward transforms, β, saved baseline state, rollout advantages	Recomputed advantages use the same historical reference across groups within a step.
Archive and execution order	Parent–child links, evaluation timestamps, gradient and optimizer events	Selected children enter the archive, ready outputs enter training, and one final update changes the policy.

Access matters: source code and internal records can expose these operations. API outputs or a final construction alone cannot establish the internal allocator or training rule.

Patent Value: Avoidability and Alternative Designs

Alternative designs change how the system explores, allocates computation, or learns.

Alternative implementation	Mechanism replaced	Technical consequence
Direct program generation, with no explicit plan stage	Frozen strategist and plan-conditioned coder	No separate plan-level sampling decision. The LLM selects an approach while implementing it.
Uniform or winner-only follow-ups	Parent-relative expected-best allocation	Uniform allocation ignores pilot differences. Winner-only allocation omits the disclosed probability-and-gain model.
Fixed model or within-group training reference	Shared coder learning or historical baseline	A fixed model does not adapt. Group centering yields zero reward advantages when all rewards tie.
Fully sequential execution	Overlap of evaluation and gradient computation	The stages remain functional but no longer overlap. This alone leaves the hierarchy and allocator intact.

Model or task substitutions can retain the same workflow. Whether an alternative avoids protection depends on the actual claims and applicable law.

Patent Value: Commercial Applications and Value

Potential customers are teams that repeatedly optimize executable solutions against a reliable evaluator.

Potential application	Candidate and evaluator	Economic value to assess
AI and cloud infrastructure	GPU kernels and compiler transformations, checked for correctness and runtime	Lower execution cost on repeated workloads and less manual kernel tuning
Scheduling and engineering optimization	Allocation heuristics and design programs, checked against constraints and cost	Better resource use and less engineering time per accepted design
Computational research	Mathematical constructions and analysis methods, checked by task-specific evaluators	Shorter search cycles and reusable records of evaluated solutions

Deployment and revenue paths

Integrate the controller into an internal R&D platform, offer private or hosted optimization, or license the technology to platform vendors.

Commercial validation

Measure time-to-target, GPU-hours per target score, and downstream operating cost. Savings must exceed generation, evaluation, and training costs.

Thank you

Hierarchical scientific discovery

Technical solution, advantageous effects, and patent value